# CyberForge AI - System Architecture & Design

## 1. System Architecture

CyberForge AI follows a modern, decoupled monolithic architecture:

- **Backend:** FastAPI (Python 3.11+). Chosen over Flask for its native asynchronous capabilities, which are crucial for long-running security tool executions (like Nmap) and non-blocking AI API calls.
- **Frontend:** HTML5 with Tailwind CSS and Vanilla JavaScript, served via Jinja2 templates. Kept lightweight to ensure maximum responsiveness.
- **Database:** SQLite (via SQLAlchemy and Pydantic) for out-of-the-box portability, with a strict ORM design allowing seamless migration to PostgreSQL for enterprise deployments.
- **AI Engine:** A custom Provider Abstraction Layer managing OpenAI, Anthropic, and Azure endpoints with exponential backoff and rate-limit handling.

## 2. Complete Folder / File Tree

```
CyberForge-AI/
├── main.py                    # FastAPI application entry point
├── requirements.txt           # Python dependencies
├── README.md                  # Project documentation
├── .env.example               # Environment variables template
├── config.example.yaml        # Application configuration template
├── docker-compose.yml         # Docker deployment configuration
├── Dockerfile                 # Application container definition
│
├── app/
│   ├── __init__.py
│   ├── config.py              # Settings management (Pydantic BaseSettings)
│   ├── routes/                # API and Web route handlers
│   ├── models/                # SQLAlchemy ORM models
│   ├── schemas/               # Pydantic validation schemas
│   ├── services/              # Core business logic
│   ├── security/              # Scope validation & authentication
│   ├── ai/                    # AI Provider abstraction layer
│   ├── parsers/               # Tool output parsers (XML, JSON, Nmap, etc.)
│   ├── integrations/          # Tool execution adapters
│   ├── evidence/              # File hashing and storage management
│   ├── reporting/             # HTML/PDF report generation engine
│   └── utils/                 # Helper functions
│
├── frontend/
│   ├── templates/             # Jinja2 HTML templates
│   ├── static/
│   │   ├── css/               # Tailwind / Custom CSS
│   │   ├── js/                # Client-side logic
│   │   └── img/               # Assets
│
├── data/                      # SQLite DB and local storage
```

```
│   ├── evidence/              # Uploaded evidence files
│   └── exports/               # Generated reports
│
└── tests/                     # Pytest suite
```

## 3. Database Schema (Core Models)

- **Case:** `id`, `name`, `client`, `type`, `operator`, `start_date`, `end_date`, `scope`, `out_of_scope`, `roe`, `status`
- **Target:** `id`, `case_id`, `identifier` (IP/Domain/URL), `type`, `status`
- **AssessmentStep:** `id`, `case_id`, `step_name`, `status`, `notes`, `timestamp`, `operator`
- **ToolRun:** `id`, `case_id`, `tool_name`, `command_executed`, `raw_output_path`, `timestamp`
- **Finding:** `id`, `case_id`, `title`, `description`, `asset`, `severity`, `cvss`, `recommendation`, `status`, `ai_reviewed`
- **Evidence:** `id`, `case_id`, `finding_id`, `filename`, `sha256`, `filepath`, `timestamp`
- **AuditLog:** `id`, `timestamp`, `operator`, `action`, `target`, `details`

## 4. API Architecture (RESTful)

- `GET /api/v1/cases` - List all cases
- `POST /api/v1/cases` - Create a new case
- `POST /api/v1/cases/{id}/tools/execute` - Trigger an authorized tool run
- `POST /api/v1/ai/analyze` - Send structured finding/evidence to AI for correlation
- `POST /api/v1/reports/generate` - Trigger HTML/PDF compilation

## 5. AI Provider Architecture

A factory pattern will manage AI providers:

- `AIProvider` (Abstract Base Class) -> `generate_completion()`, `analyze_finding()`
- `OpenAIProvider`, `AnthropicProvider`, `AzureOpenAIProvider` (Concrete Implementations)
- **Safety Interceptor:** All prompts generated by the system are appended with strict instructions to refuse autonomous exploitation and focus on reporting/correlation.

## 6. Security Model

1. **Scope Enforcement:** Before any tool execution (e.g., `nmap`), the target IP/Domain is validated against the `Case.scope` and `Case.out_of_scope` lists.
2. **Human-in-the-Loop:** AI cannot execute commands. It returns a suggested command to the UI. The Operator must click "Confirm & Execute".

3. **Data Integrity:** All uploaded evidence is hashed (SHA-256) upon upload and stored immutably.